

WORKSHOP · SESSION 1 OF 2

OAuth 2.0 & JWT — the foundations

Lessons 1–4: the mental model, the flows, and how tokens are validated.

Actors & flows

Auth Code + PKCE

Choosing a flow

JWT validation

What we cover today — Lessons 1 to 4

- | | | |
|---|--------------------------------------|--|
| 1 | OAuth Problem, Actors & Endpoints | Why OAuth exists; the four actors; tokens, scopes, consent |
| 2 | Authorization Code + PKCE | The main user flow; front vs back channel; PKCE |
| 3 | Client Credentials, Refresh & Device | Choose the right flow for the application shape |
| 4 | Token Validation & JWT Basics | Claims; decoding ≠ validation; the validation checklist |

Session 2 (date to be announced): JWS/JWE/JWKS, OAuth security controls, advanced controls (PAR, JAR, JARM, DPoP, mTLS), and FAPI 2.0.

LESSON 01

OAuth Problem, Actors & Endpoints

OAuth lets a client get limited API access on a user's behalf — without ever receiving the password.

01

A framework, not a login or a token format

OAuth 2.0 IS

- An authorization framework (RFC 6749)
- A way to get limited, scoped access via tokens
- Answering: what may this client access?

OAuth 2.0 is NOT

- Password sharing
- A single endpoint or token format
- A complete login protocol (that's OpenID Connect)
- Automatically secure — implementation matters

Framework = the model · RFCs = the spec · Flows = ways to use it.

Password sharing is the wrong shape

Give away the password

- App can do everything you can
- No way to scope to one resource
- Revoking means changing the password
- App breach exposes the whole account

Give a scoped token

- Limited to exactly what's needed
- Password never leaves the trusted service
- Revoke just this app, anytime
- Token expires on its own

The client receives permission, not the user's password.

The four actors

Resource Owner

Owns the data — the user (e.g. you on GitHub)

Client

Wants access — the app (e.g. a deployment dashboard)

Authorization Server

Authenticates the user, gets consent, issues tokens

Resource Server

Hosts the API and validates access tokens

Ask: who owns the data, who wants access, who grants it, who serves the API?

Two endpoints, two channels

Endpoint	Job	Channel
/authorize	User logs in & approves access	Front-channel (browser) — more exposed
/token	Exchange a grant for an access token	Back-channel (server-to-server) — protected

Front-channel vs back-channel is the idea behind every later security control.

Also: the access token

Represents permission, not the user's password · scoped, expiring, revocable · checked by the resource server on every call.

OAuth vs OpenID Connect

	OAuth 2.0	OpenID Connect
Purpose	Authorization	Authentication
Question	What can this client access?	Who is the user?
Main token	Access token	ID token
Example	Connect GitHub repos / Drive files	“Log in with Google”

Wants to call an API → OAuth. Wants to know who you are → OpenID Connect.

LESSON 02

Authorization Code + PKCE

The browser carries only a short-lived code; tokens come from the back-channel.
PKCE protects the exchange.

02

Authorization Code, step by step

- 1 Client redirects the browser to /authorize
- 2 User logs in and approves at the authorization server
- 3 Server redirects back with a short-lived code + state
- 4 Client posts the code to /token (back-channel)
- 5 Client receives the access token (+ refresh token)
- 6 Client calls the API with the access token

Authorization request parameters

Parameter	Purpose
response_type=code	Ask for an authorization code
client_id	Identify the client
redirect_uri	Where the browser returns (must match registration)
scope	Permissions requested
state	Bind the callback to a request the client started (CSRF)
code_challenge (+ _method=S256)	PKCE challenge for the code exchange

A stolen code should not be enough

Without PKCE

- Attacker intercepts the authorization code
- Sends it to /token
- If the client is public, may get tokens

With PKCE

- Client first sends code_challenge (a hash)
- Later proves it with the code_verifier at /token
- Attacker has the code but not the verifier → fails

```
code_challenge = BASE64URL( SHA256(code_verifier) ) // S256, one-way
```

state vs redirect_uri vs PKCE

Check	Checked by	Protects
state	Client	Callbacks the client did not start (CSRF)
redirect_uri	Authorization server	Binds the code to the original request target
code_verifier / challenge	Token endpoint	Proves the redeemer knows the original secret

They solve different problems — a modern browser flow uses all three.

LESSON 03

Client Credentials, Refresh & Device

OAuth is not one flow — different trust situations need different flows.

03

Three more flows, three problems

Client Credentials

Machine-to-machine — no user, no browser, no consent. Token represents the service.

Refresh Token

Renew a short-lived access token without sending the user through the flow again.

Device Authorization

Limited-input devices (CLI, TV): user approves on a second device while the client polls.

Rotation makes reuse detectable

Step	What happens
Issue	Client gets access token AT1 + refresh token RT1
Refresh	AT1 expires; client sends RT1 to /token
Rotate	Server returns AT2 + RT2, marks RT1 used
Reuse seen	If RT1 appears again → treat as theft, revoke the family

An access token opens the door once; a refresh token can keep making new keys — store it well.

Choose the flow

Flow	User?	Redirect?	Main use	Today?
Auth Code + PKCE	Yes	Yes	Web, SPA, mobile with a user	Recommended
Client Credentials	No	No	Service-to-service APIs	Recommended
Refresh Token	Sometimes	No	Renew an access token	Recommended w/ controls
Device Authorization	Yes	Partial	CLI / TV / limited input	When appropriate
Implicit / Password	Yes	—	Legacy	Avoid

LESSON 04

Token Validation & JWT Basics

A JWT is only a format. Security comes from validation — decoding is not validation.

04

JWT: header . payload . signature

decoded payload

```
{
  "iss": "https://auth.example.com",
  "sub": "user_123",
  "aud": "https://api.example.com",
  "exp": 1760000000,
  "scope": "repo.read repo.write"
}
```

Claims that drive validation

- iss — do we trust this issuer?
- aud — is this token meant for this API?
- exp / nbf — is it valid right now?
- scope — enough permission for this action?

A signed JWT is only base64url-encoded — anyone holding it can read the payload.

What the resource server must check

Validation checklist

- Signature verifies with a trusted key
- alg is expected; kid is a trusted key
- iss trusted · aud is this API
- exp / nbf valid in time
- scope sufficient · correct token type

Reject for the right reason

- 401 Unauthorized — missing, expired, malformed, invalid token
- 403 Forbidden — valid token, but not enough scope
- Opaque (non-JWT) token? Call the introspection endpoint

COMING IN SESSION 2 · DATE TO BE ANNOUNCED

Where Session 2 goes next

Lesson 5

JWS, JWE, JWK / JWKS — signing vs encryption, publishing & rotating keys

Lesson 6

OAuth security controls — deprecated flows; attacks mapped to mitigations

Lesson 7

Advanced: PAR, JAR, JARM, DPoP, mTLS — hardening requests & token possession

Lesson 8

FAPI 2.0 — strict OAuth for high-value APIs (open banking, payments, health)

SESSION 1 · YOU'RE READY WHEN...

Ready for Session 2

- ✓ Explain OAuth in two minutes without mentioning JWT
- ✓ Draw Authorization Code + PKCE from memory
- ✓ Choose the right flow for a given application shape
- ✓ List the JWT validation checks and why decoding isn't validation